

Uber Supply Demand Gap Analysis

Project Objective

The objective of this project is to analyze the Uber Supply-Demand Gap dataset and identify the major reasons behind ride cancellations and unavailability of cars. The analysis is performed using Python to understand customer demand patterns and provide business recommendations.

Tools Used

- Python
- Pandas
- NumPy
- Matplotlib

Dataset

Uber Request Data.csv

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline
```

```
In [3]: df = pd.read_csv("Uber Request Data.csv")

print("Dataset Loaded Successfully")
```

Dataset Loaded Successfully

```
In [4]: df.head()
```

Out[4]:

	Request id	Pickup point	Driver id	Status	Request timestamp	Drop timestamp
0	619	Airport	1.0	Trip Completed	11/07/16 11:51	11/07/16 13:00
1	867	Airport	1.0	Trip Completed	11/07/16 17:57	11/07/16 18:47
2	1807	City	1.0	Trip Completed	12/07/16 9:17	12/07/16 9:58
3	2532	Airport	1.0	Trip Completed	12/07/16 21:08	12/07/16 22:03
4	3112	City	1.0	Trip Completed	13/07/16 8:33	13/07/16 9:25

In [5]:

```
print("Dataset Shape")

rows, columns = df.shape

print("Rows :", rows)
print("Columns :", columns)
```

Dataset Shape
Rows : 6745
Columns : 6

In [6]:

```
df.info()
df.dtypes
df.describe(include="all")
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6745 entries, 0 to 6744
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Request id            6745 non-null  int64
1   Pickup point          6745 non-null  str
2   Driver id             4095 non-null  float64
3   Status                6745 non-null  str
4   Request timestamp     6745 non-null  str
5   Drop timestamp        2831 non-null  str
dtypes: float64(1), int64(1), str(4)
memory usage: 574.1 KB
```

Out[6]:

	Request id	Pickup point	Driver id	Status	Request timestamp	Drop timestamp
count	6745.000000	6745	4095.000000	6745	6745	2831
unique	NaN	2	NaN	3	4016	2282
top	NaN	City	NaN	Trip Completed	15/07/16 19:19	13/07/16 8:53
freq	NaN	3507	NaN	2831	8	5
mean	3384.644922	NaN	149.501343	NaN	NaN	NaN
std	1955.099667	NaN	86.051994	NaN	NaN	NaN
min	1.000000	NaN	1.000000	NaN	NaN	NaN
25%	1691.000000	NaN	75.000000	NaN	NaN	NaN
50%	3387.000000	NaN	149.000000	NaN	NaN	NaN
75%	5080.000000	NaN	224.000000	NaN	NaN	NaN
max	6766.000000	NaN	300.000000	NaN	NaN	NaN

Data Cleaning

In this section, the dataset is checked for missing values, duplicate records and incorrect data types before performing analysis.

```
In [8]: df.isnull().sum()
```

```
Out[8]: Request id          0
Pickup point          0
Driver id          2650
Status              0
Request timestamp     0
Drop timestamp       3914
dtype: int64
```

```
In [9]: print("Duplicate Records :",df.duplicated().sum())
```

```
Duplicate Records : 0
```

```
In [10]: df['Request timestamp'] = pd.to_datetime(df['Request timestamp'],dayfirst=True)
df['Drop timestamp'] = pd.to_datetime(df['Drop timestamp'],dayfirst=True,
```

```
In [11]: df['Hour'] = df['Request timestamp'].dt.hour

df['Day'] = df['Request timestamp'].dt.day

df['Weekday'] = df['Request timestamp'].dt.day_name()

df['Month'] = df['Request timestamp'].dt.month_name()
```

```
In [12]: def time_slot(hour):

    if hour>=0 and hour<5:
        return "Early Morning"

    elif hour>=5 and hour<10:
        return "Morning"

    elif hour>=10 and hour<17:
        return "Afternoon"

    elif hour>=17 and hour<21:
        return "Evening"

    else:
        return "Night"

df['Time Slot']=df['Hour'].apply(time_slot)

df.head()
```

```
Out[12]:
```

	Request id	Pickup point	Driver id	Status	Request timestamp	Drop timestamp	Hour	Day	Week
0	619	Airport	1.0	Trip Completed	2016-07-11 11:51:00	2016-07-11 13:00:00	11	11	Moi
1	867	Airport	1.0	Trip Completed	2016-07-11 17:57:00	2016-07-11 18:47:00	17	11	Moi
2	1807	City	1.0	Trip Completed	2016-07-12 09:17:00	2016-07-12 09:58:00	9	12	Tue
3	2532	Airport	1.0	Trip Completed	2016-07-12 21:08:00	2016-07-12 22:03:00	21	12	Tue
4	3112	City	1.0	Trip Completed	2016-07-13 08:33:00	2016-07-13 09:25:00	8	13	Wedne

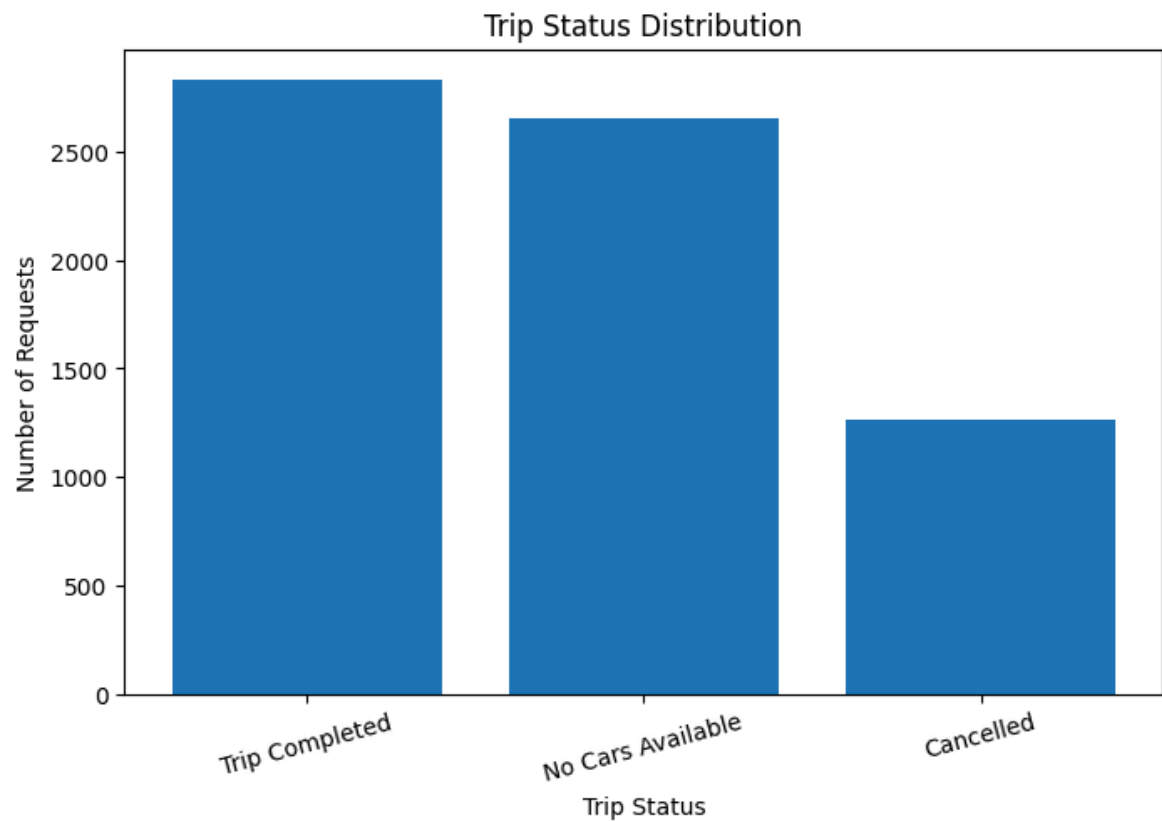
Exploratory Data Analysis (EDA)

In this section, different visualizations are created to understand customer demand, trip status, pickup points, request timings, and supply-demand gap.

```
In [13]: status = df['Status'].value_counts()

plt.figure(figsize=(8,5))
plt.bar(status.index, status.values)
plt.title("Trip Status Distribution")
plt.xlabel("Trip Status")
plt.ylabel("Number of Requests")
plt.xticks(rotation=15)
plt.show()

status
```



```
Out[13]: Status
Trip Completed      2831
No Cars Available   2650
Cancelled           1264
Name: count, dtype: int64
```

Insight

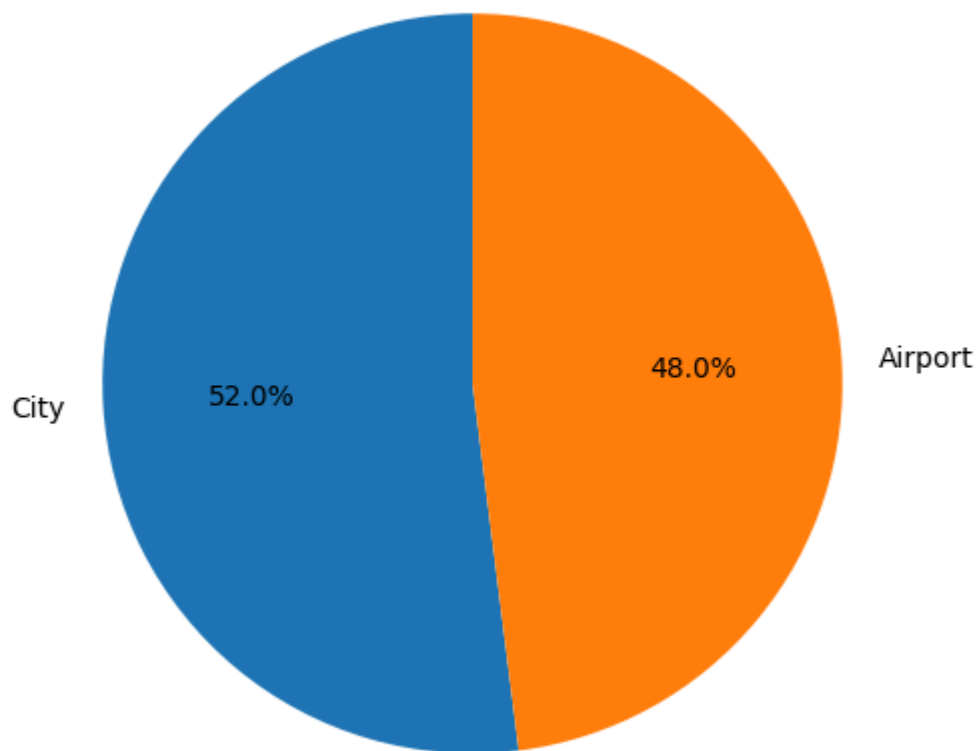
- Trip Completed has the highest number of requests.
- A significant number of rides were cancelled.
- Many customers could not get a cab due to unavailability.

```
In [14]: pickup = df['Pickup point'].value_counts()

plt.figure(figsize=(6,6))
plt.pie(pickup.values,
        labels=pickup.index,
        autopct='%1.1f%%',
        startangle=90)

plt.title("Requests by Pickup Point")
plt.show()
```

Requests by Pickup Point



Insight

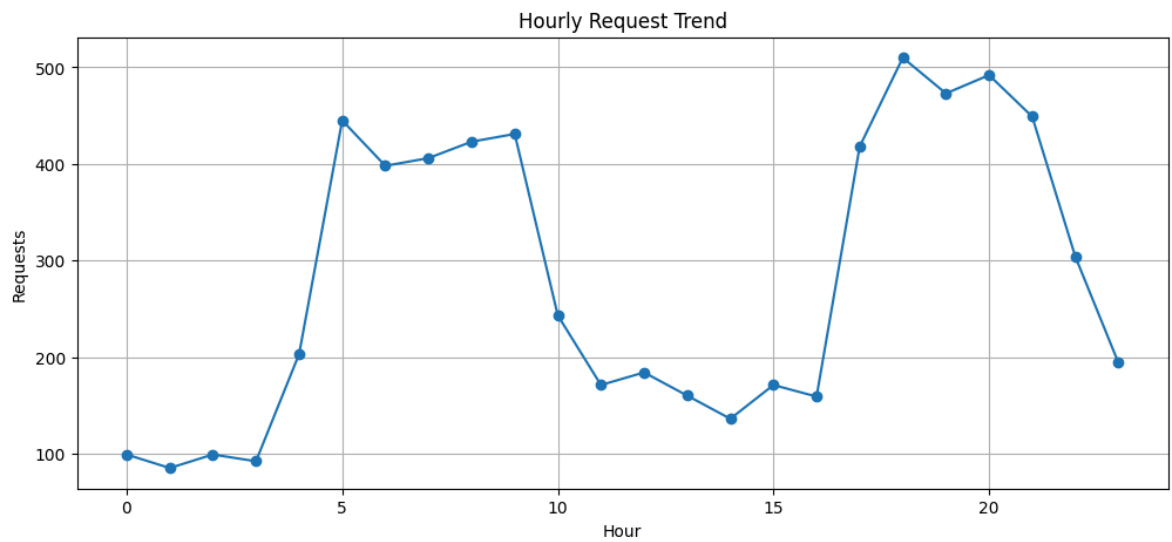
- Airport and City contribute most ride requests.
- Demand is distributed between both pickup locations.

```
In [17]: hour = df['Hour'].value_counts().sort_index()

plt.figure(figsize=(12,5))
plt.plot(hour.index,
         hour.values,
         marker='o')

plt.title("Hourly Request Trend")
plt.xlabel("Hour")
plt.ylabel("Requests")
plt.grid(True)

plt.show()
```



Insight

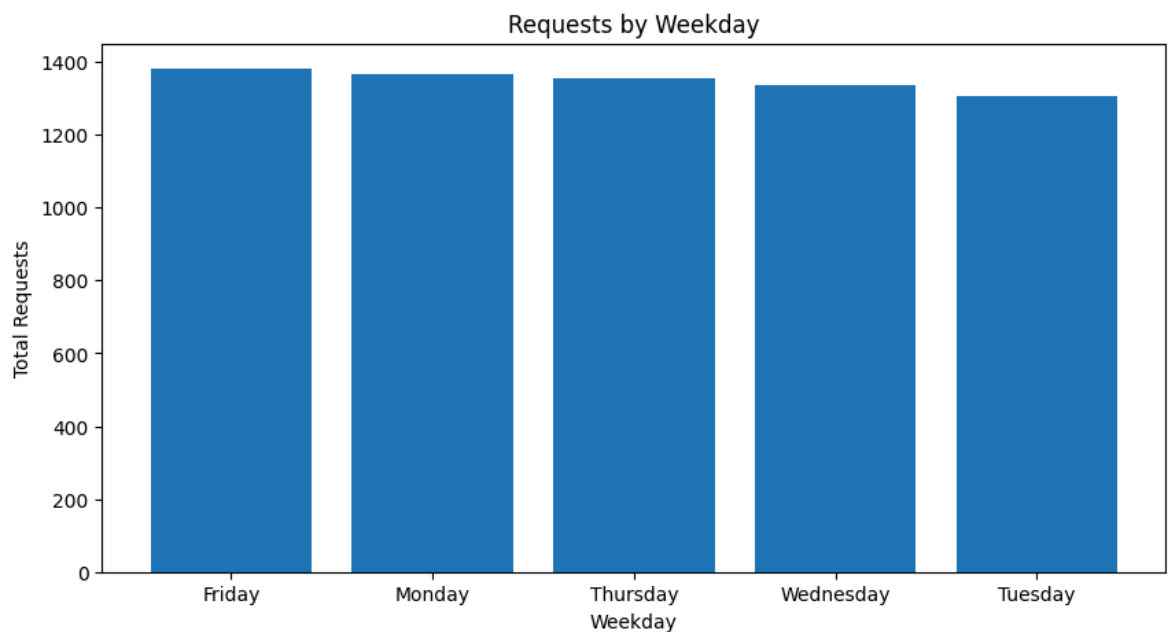
- Ride demand changes throughout the day.
- Peak demand occurs during office commute hours.

```
In [18]: weekday = df['Weekday'].value_counts()
```

```
plt.figure(figsize=(10,5))
plt.bar(weekday.index,
        weekday.values)

plt.title("Requests by Weekday")
plt.xlabel("Weekday")
plt.ylabel("Total Requests")

plt.show()
```



Insight

- Demand varies across weekdays.

- Certain weekdays have higher ride requests than others.

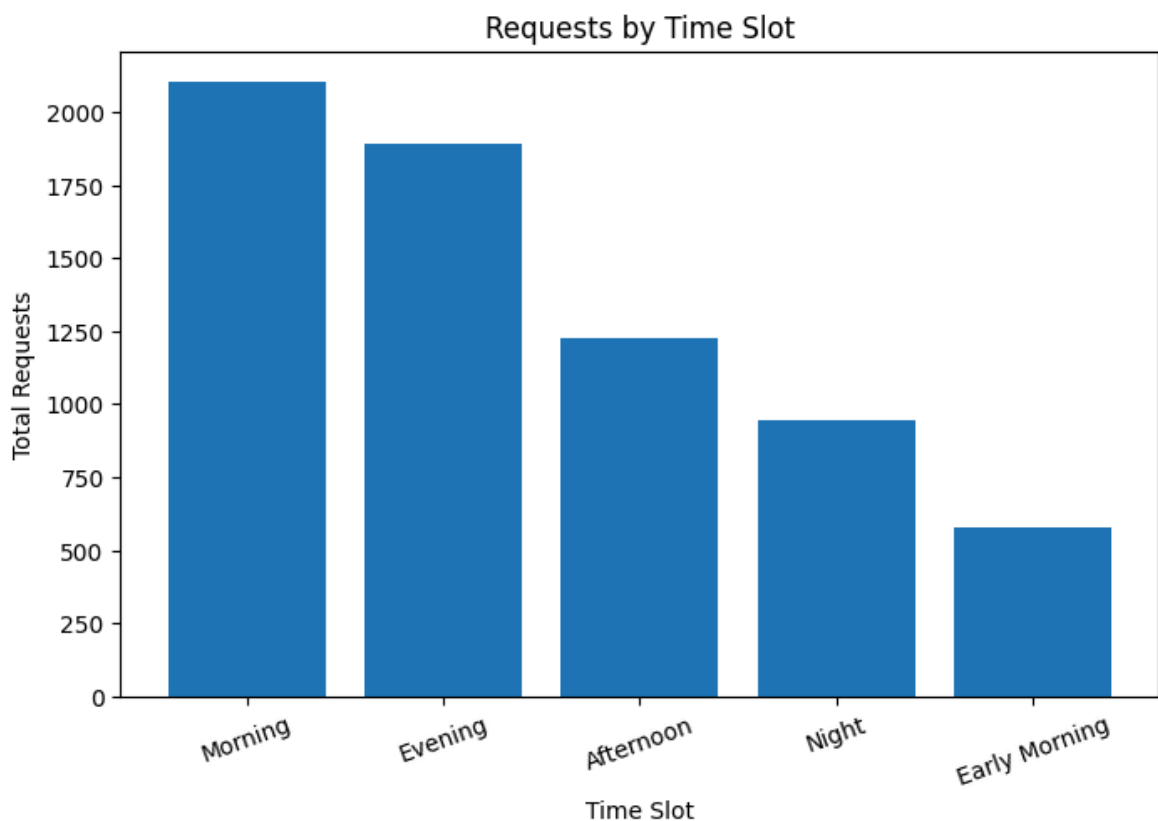
```
In [19]: timeslot = df['Time Slot'].value_counts()

plt.figure(figsize=(8,5))
plt.bar(timeslot.index,
        timeslot.values)

plt.title("Requests by Time Slot")
plt.xlabel("Time Slot")
plt.ylabel("Total Requests")

plt.xticks(rotation=20)

plt.show()
```



Insight

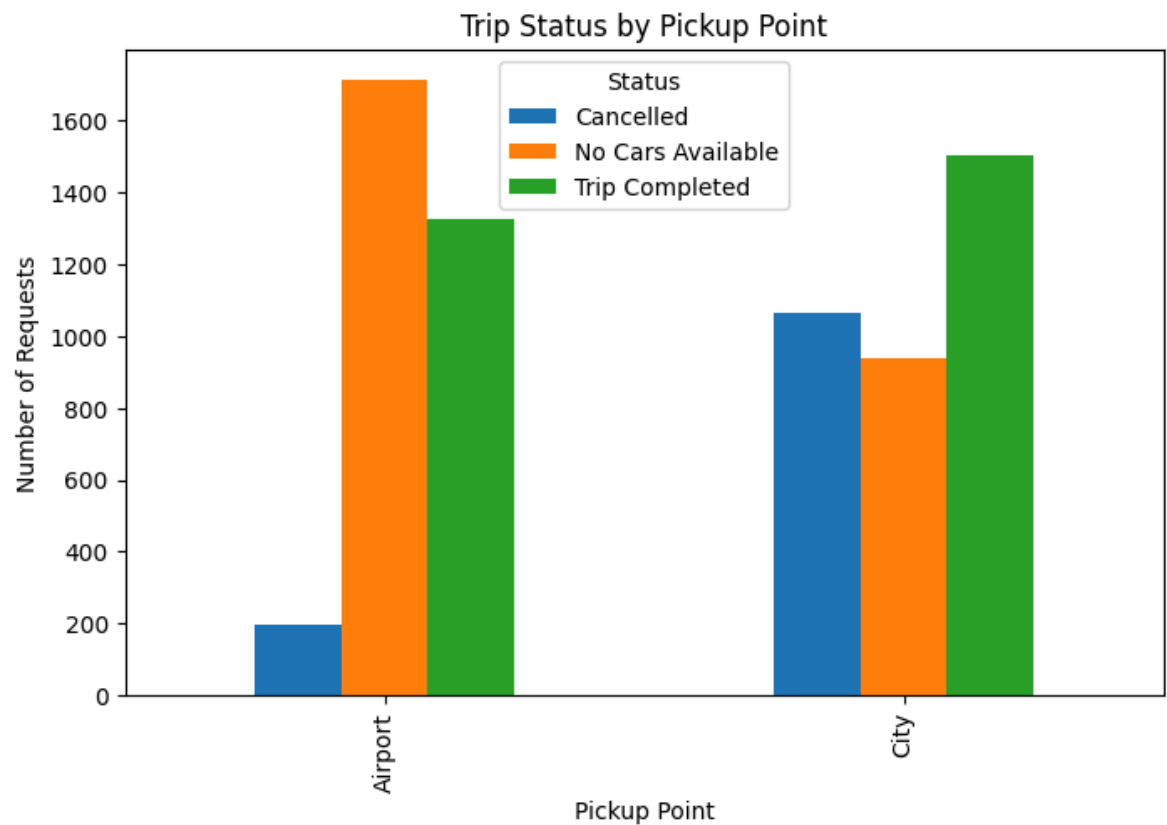
- Morning and Evening time slots generally experience higher demand.
- Supply-demand gaps become more visible during busy periods.

```
In [20]: pivot = pd.crosstab(df['Pickup point'],
                             df['Status'])

pivot.plot(kind='bar',
           figsize=(8,5))

plt.title("Trip Status by Pickup Point")
plt.xlabel("Pickup Point")
plt.ylabel("Number of Requests")

plt.show()
```

Insight

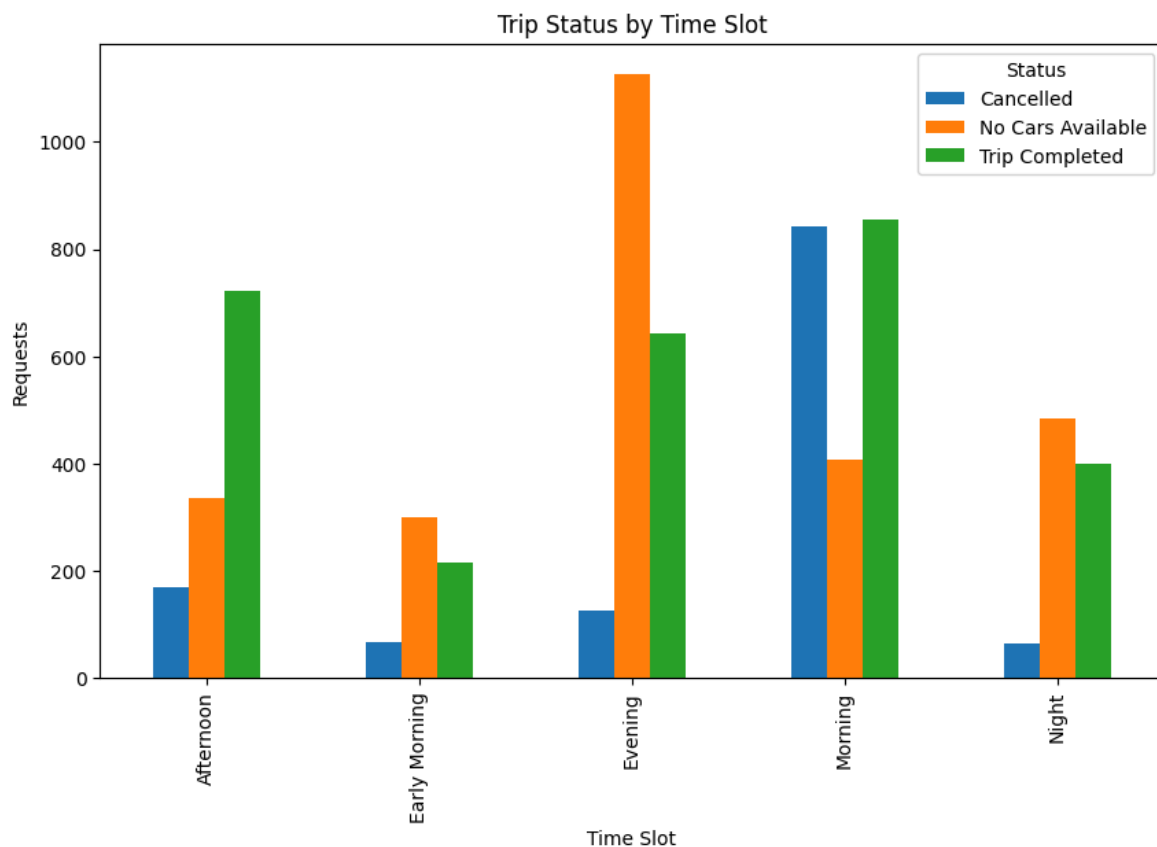
- The Airport experiences a larger number of "No Cars Available" cases.
- The City has comparatively more cancelled trips.

```
In [21]: pivot2 = pd.crosstab(df['Time Slot'],
                             df['Status'])

pivot2.plot(kind='bar',
            figsize=(10,6))

plt.title("Trip Status by Time Slot")
plt.xlabel("Time Slot")
plt.ylabel("Requests")

plt.show()
```



Insight

- Early Morning and Night show higher supply-demand gaps.
- Driver cancellations increase during busy periods.

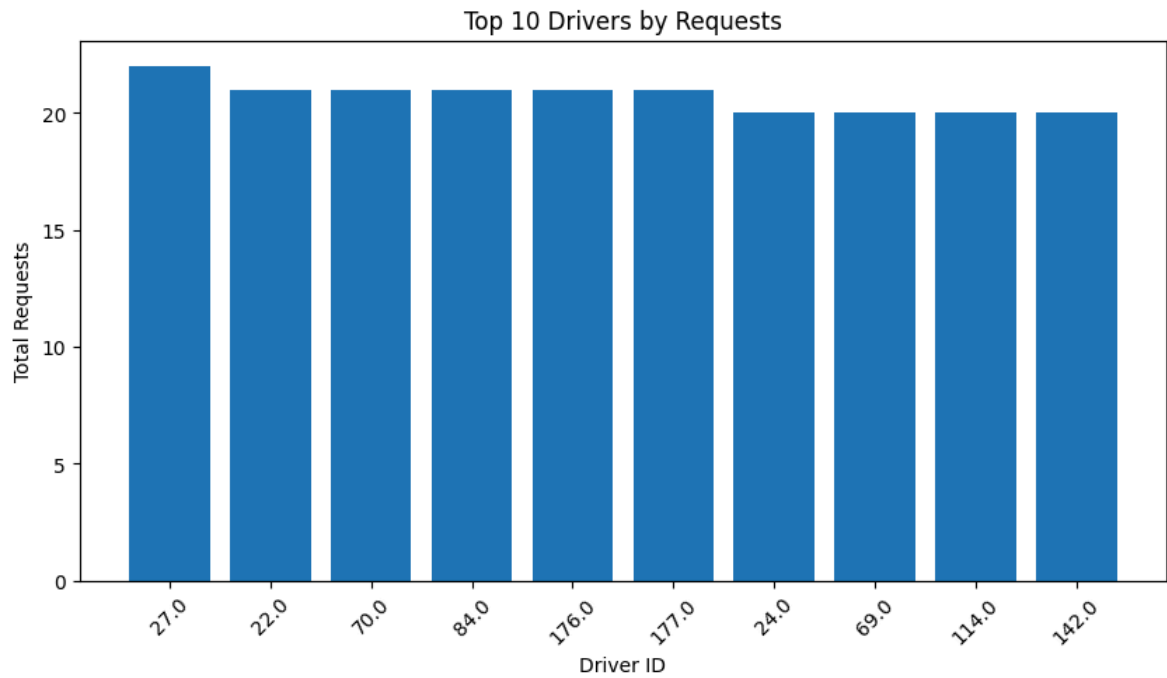
```
In [22]: driver = df[df['Driver id'].notna()]['Driver id'].value_counts().head(10)

plt.figure(figsize=(10,5))
plt.bar(driver.index.astype(str),
        driver.values)

plt.title("Top 10 Drivers by Requests")
plt.xlabel("Driver ID")
plt.ylabel("Total Requests")

plt.xticks(rotation=45)

plt.show()
```



Insight

- Some drivers complete significantly more trips than others.
- Driver workload is not equally distributed.

Conclusion

Key Findings

- Most requests were successfully completed.
- A considerable number of requests were cancelled.
- Airport experiences more "No Cars Available" situations.
- Morning and Evening are the busiest periods.
- Driver availability affects customer satisfaction.
- Supply-demand gap exists mainly during peak demand periods.

Recommendations

- Increase driver availability during peak hours.
- Introduce incentives for drivers during high-demand periods.
- Improve ride allocation at Airport pickup points.
- Reduce driver cancellations through better scheduling and incentive programs.

In []: